Some basics about RoPE

- *(prev)*
- Genesis
- **What is position encodings? Why do we use them?**
  - Unlike RNNs, transformers themselves don't contain any positional information. However, the meaning of a sentence in modern languages (*some old languages, because of the high fusional or agglutinative nature, may not rely on order too much*) heavily depends on the order of words, or tokens. **Thus, we need to find a way to also embed position info into the data.**
- **Absolute PE**
  - Classically, people use absolute PE to inject the position info. Formally $\mathbf{X}' = \mathbf{X} + \mathbf{P}$, where $\mathbf{X}'$ is the input vector after PE, $\mathbf{X}$ is the vector after the embedding, and $\mathbf{P}$ is the position encoding vector, where $p_i \in \mathbf{P}$ is the position encoding of the token on index i. This vector is generated from an **absolute index dependent** function, $f(i)$.
  - A commonly used way to calculate this is sinusoidal PE. Formally:
    $$f(i)_{2k} = \sin\left(\frac{i}{10000^{\frac{2k}{d}}}\right)$$
    $$f(i)_{2k+1} = \cos\left(\frac{i}{10000^{\frac{2k}{d}}}\right)$$
    , where $i$is the index, $k$is the dimension index, and $d$is the dimension of the embedding.
  - However, it is found that in languages, **the relative position** is sometimes even more important than the absolute position. For example,

    ```
    The cat sat on the mat.
    On the mat, the cat sat.
    ```

  In sentence 1 above, "on" is in the middle of the sentence and represents the relationship between "sat" and "mat". In sentence 2, we change the order (the absolute position) of the words, but "on", at the beginning of the sentence, still represents the relationship between "mat" and "sat".
- **RoPE, a commonly-used way to achieve the relative position encoding**
  - $R(n) = \begin{bmatrix} \cos(n\theta) & -\sin(n\theta) \\ \sin(n\theta) & \cos(n\theta) \end{bmatrix}$ is a rotation matrix, which has the following property:

  $R(n)\ R(m)^{-1} = R(n)\ R(m)^{\top} = R(n-m)$ (*proof*)
  - $R(n)$and $R(m)$ are two rotation matrices that **only** depend on $n$and m, the positions.
  - $\theta_j$is the rotation angle, where $\theta_j = \alpha_j \cdot n$; $j$is the position of the is the index of the **2D subspace** within the hidden dimension, i.e., each rotation is applied independently to pairs of dimensions in the query and key vectors (for example, the columns of K and Q matrices). Normally, we let $\alpha_j = \frac{1}{10000^{2j/d}}$, where d is the dimension of the hidden state (i.e., $d = d_{\text{hidden}}$).
  - Because attention heads find the relationship between two words in some semantic spaces via $QK^{\top}$step, we want to inject the position info just before this step. For the query token with position $n$and key token with position m, we apply:

  $q_m = R_\theta(m)W^Q x_m,\ k_n = R_\theta(n)W^K x_n$

$$R\mathbf{v} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x\cos\theta - y\sin\theta \\ x\sin\theta + y\cos\theta \end{bmatrix}.$$

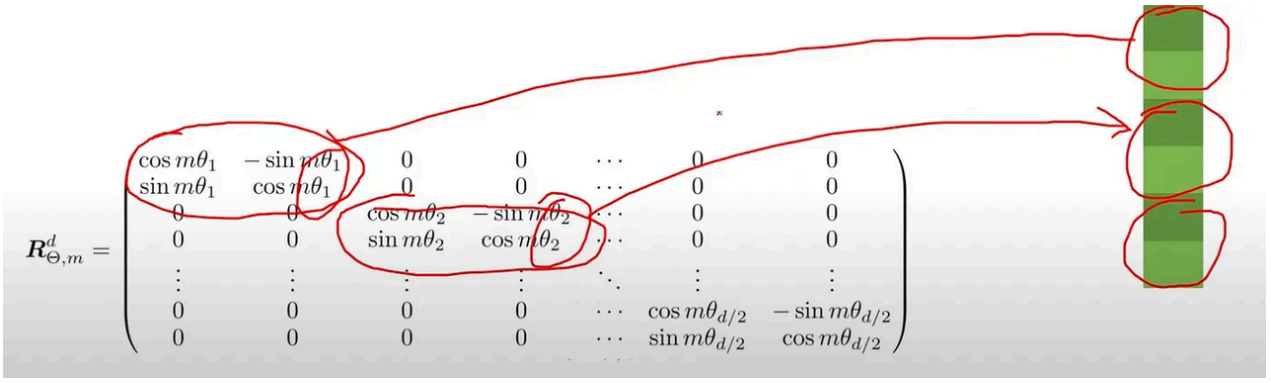
(Notice here the x vector is a vertical one; so when we group the elements, there is actually a "transpose")

Apply $QK^\top$ to this very particular pair of tokens based on what we have proved (here we should use $Q^\top K$actually, as the $x$is being transposed)

$$\text{Attention}(m,n) = q_m^\top k_n = (R_\theta(m)W^Q x_m)^\top (R_\theta(n)W^K x_n) = x_m^\top W^{Q\top} R_\theta(n-m) W^K x_n$$

Here we can see that $R_\theta(n-m)$, the relative position based on $n-m$, has been "injected" to the attention score.

- On a micro-scale, as the definition of the rotation matrix given above is a 2×2 matrix, we group the element in vector x_m and x_n into groups with 2 elements, *(implicitely)* transpose it and calculate the product.
- Each group will have a different rotation matrix as $a_j$depends on j. We diagonally put these rotation matrices into a large matrix, so we can finish this process once for all. The relationship between each group and the small matrices in the large matrix is shown below:



(credits)

- **Why don't we directly plug the position into the K, Qmatrices?**
  - Well, the indices will increase linearly. If a sequence is very long, say 10000 tokens, then the attention scores will be very large in some cases (just because of the large indices, not the actual closeness of the relationship of those two tokens!) This will decrease the stability of training.
  - With RoPE, α_j, θ_j, and $\cos, \sin$ functions will limit the growth of the values while maintaining the position information.
- **Work Cited**
  - Understanding Rotary Positional Encoding
  - Rotary Embeddings: A Relative Revolution
  - ROFORMER: ENHANCED TRANSFORMER WITH ROTARY POSITION EMBEDDING